Python Module 06
Import Mysteries

Complete beginner study guide + subject-based code

This guide follows the supplied Module 06 subject: packages, import styles, absolute vs relative imports, and circular dependencies.

Important: The subject explicitly says you must understand AI-assisted code and be able to explain it during evaluation. Use this as a study copy, then rebuild parts yourself from memory.

1. What the subject requires

- Python 3.10 or later.
- flake8 coding style.
- Comprehensive type annotations, checked with mypy.
- Built-in types, collections, and built-in functions are allowed, except eval() and exec().
- Only imports from files/modules created in this project are allowed.
- Do not modify sys.path.
- Place the expected file tree at the root of the Git repository.
- Be ready to explain imports, demonstrate import styles, and modify the project during evaluation.

Two intentional failures: ft_alembic_4.py must fail when it tries to access alchemy.create_earth(), because create_earth is not exposed by alchemy. ft_kaboom_1.py must fail because the dark modules intentionally create a circular import.

2. New subjects to learn first

Module: A single .py file. Example: elements.py.

Package: A folder of Python modules. __init__.py defines the package interface.

import module: Imports the module object. You use module.function().

from module import name: Imports one selected function/name directly.

Nested package import: Reaches code inside folders such as alchemy.transmutation.recipes.

Alias: Renames an imported function, for example healing_potion as heal.

Absolute import: Starts from a top-level package/module name, such as import elements.

Relative import: Uses dots inside a package, such as from ..potions import strength_potion.

Package interface: Names imported by __init__.py become easy to access through the package.

Circular import: Module A imports B while B imports A before either has finished loading.

Local import: An import placed inside a function. It can delay loading and break a circular dependency.

Type annotation: A hint such as ingredients: str or -> str that mypy can check.

__name__ guard: Runs main() only when a test file is executed directly.

3. Final file tree

```
.
|-- alchemy
|   |-- __init__.py
|   |-- elements.py
|   |-- grimoire
|   |   |-- __init__.py
|   |   |-- dark_spellbook.py
|   |   |-- dark_validator.py
|   |   |-- light_spellbook.py
|   |   `-- light_validator.py
|   |-- potions.py
|   `-- transmutation
|       |-- __init__.py
```

Study the imports, not only the final output.

```
|       `-- recipes.py
|-- elements.py
|-- ft_alembic_0.py ... ft_alembic_5.py
|-- ft_distillation_0.py
|-- ft_distillation_1.py
|-- ft_kaboom_0.py
|-- ft_kaboom_1.py
|-- ft_transmutation_0.py
|-- ft_transmutation_1.py
`-- ft_transmutation_2.py
```

4. Part I - The Alembic

Goal: learn the difference between importing a whole module, importing one name, and exposing selected names through a package.

elements.py

What it does: Creates the top-level fire and water elements.

New thing to learn: A normal Python module and functions that return strings.

```
def create_fire() -> str:
    """Return the message for creating fire."""
    return "Fire element created"


def create_water() -> str:
    """Return the message for creating water."""
    return "Water element created"
```

alchemy/elements.py

What it does: Creates earth and air inside the alchemy package.

New thing to learn: The same filename can exist in a different package path.

```
def create_earth() -> str:
    """Return the message for creating earth."""
    return "Earth element created"


def create_air() -> str:
    """Return the message for creating air."""
    return "Air element created"
```

alchemy/__init__.py

What it does: Defines the final public interface of alchemy. create_air is exposed, create_earth is intentionally not exposed. It also exposes later project features.

New thing to learn: __init__.py, aliases, __all__, and package interfaces.

Beginner note: __all__ is a list of intended public names. The key reason create_earth is unavailable as alchemy.create_earth is simply that __init__.py never imports it.

```
from .elements import create_air
from .potions import healing_potion as heal
from .potions import strength_potion
from . import transmutation

__all__ = ["create_air", "strength_potion", "heal", "transmutation"]
```

ft_alembic_0.py

What it does: Imports elements as a module and calls elements.create_fire().

New thing to learn: import module.

```
import elements


def main() -> None:
    """Demonstrate a normal module import."""
    print("=== Alembic 0 ===")
    print("Using: 'import ...' structure to access elements.py")
    print(f"Testing create_fire: {elements.create_fire()}")


if __name__ == "__main__":
    main()
```

ft_alembic_1.py

What it does: Imports create_water directly from elements.

New thing to learn: from module import name.

```
from elements import create_water


def main() -> None:
    """Demonstrate importing one name from a module."""
    print("=== Alembic 1 ===")
    print("Using: 'from ... import ...' structure to access elements.py")
    print(f"Testing create_water: {create_water()}")


if __name__ == "__main__":
    main()
```

ft_alembic_2.py

What it does: Imports alchemy.elements and calls the function through the full module path.

New thing to learn: Importing a module inside a package.

```
import alchemy.elements


def main() -> None:
    """Demonstrate importing a module inside a package."""
    print("=== Alembic 2 ===")
    print("Accessing alchemy/elements.py using 'import ...' structure")
    earth = alchemy.elements.create_earth()
    print(f"Testing create_earth: {earth}")


if __name__ == "__main__":
    main()
```

ft_alembic_3.py

What it does: Imports create_air directly from alchemy.elements.

New thing to learn: from package.module import name.

```
from alchemy.elements import create_air


def main() -> None:
```

```
    """Demonstrate importing one name from a package module."""
    print("=== Alembic 3 ===")
    print(
        "Accessing alchemy/elements.py using "
        "'from ... import ...' structure"
    )
    print(f"Testing create_air: {create_air()}")


if __name__ == "__main__":
    main()
```

ft_alembic_4.py

What it does: Imports alchemy, successfully calls create_air, then intentionally fails on create_earth.

New thing to learn: A package only exposes names that its __init__.py makes available.

```
import alchemy


def main() -> None:
    """Show the public package interface and a deliberately hidden name."""
    print("=== Alembic 4 ===")
    print("Accessing the alchemy module using 'import alchemy'")
    print(f"Testing create_air: {alchemy.create_air()}")
    print("Now show that not all functions can be reached")
    print("This will raise an exception!")
    print(f"Testing the hidden create_earth: {alchemy.create_earth()}")


if __name__ == "__main__":
    main()
```

ft_alembic_5.py

What it does: Imports create_air directly from the alchemy package interface.

New thing to learn: from package import exposed_name.

```
from alchemy import create_air


def main() -> None:
    """Demonstrate importing a name exposed by a package."""
    print("=== Alembic 5 ===")
    print("Accessing the alchemy module using 'from alchemy import ...'")
    print(f"Testing create_air: {create_air()}")


if __name__ == "__main__":
    main()
```

What to remember for evaluation: import elements gives you a module object. from elements import create_water gives you the function name directly. import alchemy uses the interface defined in alchemy/__init__.py.

5. Part II - Distillation

Goal: make one module use functions located in other files, including a file outside the package and another file inside the package.

alchemy/potions.py

What it does: Builds healing and strength potion strings from the four element functions.

New thing to learn: Nested imports and combining absolute and relative paths.

Beginner note: import elements reaches the root elements.py. from .elements reaches alchemy/elements.py. The single dot means "this package".

```
import elements

from .elements import create_air, create_earth


def healing_potion() -> str:
    """Brew a potion from the earth and air elements."""
    earth = create_earth()
    air = create_air()
    return f"Healing potion brewed with '{earth}' and '{air}'"


def strength_potion() -> str:
    """Brew a potion from the fire and water elements."""
    fire = elements.create_fire()
    water = elements.create_water()
    return f"Strength potion brewed with '{fire}' and '{water}'"
```

ft_distillation_0.py

What it does: Imports both potion functions directly from alchemy.potions.

New thing to learn: Direct access to a nested module.

```
from alchemy.potions import healing_potion, strength_potion


def main() -> None:
    """Use potion functions directly from their module."""
    print("=== Distillation 0 ===")
    print("Direct access to alchemy/potions.py")
    print(f"Testing strength_potion: {strength_potion()}")
    print(f"Testing healing_potion: {healing_potion()}")


if __name__ == "__main__":
    main()
```

ft_distillation_1.py

What it does: Imports only alchemy, then uses strength_potion and the heal alias exposed by __init__.py.

New thing to learn: Aliases and a clean package interface.

```
import alchemy


def main() -> None:
    """Use potion functions exposed by the alchemy package."""
    print("=== Distillation 1 ===")
    print("Using: 'import alchemy' structure to access potions")
    print(f"Testing strength_potion: {alchemy.strength_potion()}")
    print(f"Testing heal alias: {alchemy.heal()}")


if __name__ == "__main__":
    main()
```

6. Part III - The Great Transmutation

Goal: understand absolute and relative imports. The subject requires recipes.py to contain at least one of each.

alchemy/transmutation/recipes.py

What it does: Creates the lead-to-gold recipe using air, a strength potion, and fire.

New thing to learn: Absolute imports versus relative imports.

Beginner note: Absolute: import elements and from alchemy.elements import create_air. Relative: from ..potions import strength_potion. Two dots mean "go up one package level from transmutation to alchemy".

```
import elements

from alchemy.elements import create_air
from ..potions import strength_potion


def lead_to_gold() -> str:
    """Return the recipe that turns lead into gold."""
    air = create_air()
    strength = strength_potion()
    fire = elements.create_fire()
    return (
        "Recipe transmuting Lead to Gold: "
        f"brew '{air}' and '{strength}' mixed with '{fire}'"
    )
```

alchemy/transmutation/__init__.py

What it does: Exposes lead_to_gold directly through alchemy.transmutation.

New thing to learn: Using __init__.py to shorten a long import path.

```
from .recipes import lead_to_gold

__all__ = ["lead_to_gold"]
```

ft_transmutation_0.py

What it does: Imports the recipes file through its complete package path.

New thing to learn: Full nested module paths.

```
import alchemy.transmutation.recipes


def main() -> None:
    """Access the recipes file through its full package path."""
    print("=== Transmutation 0 ===")
    print("Using file alchemy/transmutation/recipes.py directly")
    recipe = alchemy.transmutation.recipes.lead_to_gold()
    print(f"Testing lead to gold: {recipe}")


if __name__ == "__main__":
    main()
```

ft_transmutation_1.py

What it does: Imports alchemy.transmutation with a shorter local alias.

New thing to learn: Module aliases using as.

```
import alchemy.transmutation as transmutation


def main() -> None:
    """Import the transmutation package and use its public function."""
    print("=== Transmutation 1 ===")
    print("Import transmutation module directly")
    print(f"Testing lead to gold: {transmutation.lead_to_gold()}")


if __name__ == "__main__":
    main()
```

ft_transmutation_2.py

What it does: Imports only alchemy, then reaches alchemy.transmutation.lead_to_gold().

New thing to learn: Re-exporting a subpackage from a parent package.

```
import alchemy


def main() -> None:
    """Import only alchemy and reach its transmutation package."""
    print("=== Transmutation 2 ===")
    print("Import alchemy module only")
    recipe = alchemy.transmutation.lead_to_gold()
    print(f"Testing lead to gold: {recipe}")


if __name__ == "__main__":
    main()
```

Absolute or relative?: Use absolute imports when you want the full project path to be obvious. Use relative imports for closely related modules inside the same package. Keep the style consistent and avoid very confusing chains of dots.

7. Part IV - Avoid the Explosion

Goal: understand why circular imports happen and how delaying one import can break the cycle.

alchemy/grimoire/light_spellbook.py

What it does: Defines allowed light ingredients and records or rejects a spell after validation.

New thing to learn: One-way top-level dependency: spellbook imports validator.

```
from .light_validator import validate_ingredients


def light_spell_allowed_ingredients() -> list[str]:
    """Return ingredients accepted by light magic."""
    return ["earth", "air", "fire", "water"]


def light_spell_record(spell_name: str, ingredients: str) -> str:
    """Record a light spell when at least one ingredient is allowed."""
    validation = validate_ingredients(ingredients)
    if validation.endswith(" - VALID"):
        return f"Spell recorded: {spell_name} ({validation})"
    return f"Spell rejected: {spell_name} ({validation})"
```

alchemy/grimoire/light_validator.py

What it does: Checks whether the input text contains at least one allowed light ingredient, ignoring case.

New thing to learn: Local imports as a simple circular-import fix.

Beginner note: The import of light_spell_allowed_ingredients happens inside validate_ingredients(). By the time the function runs, light_spellbook has finished loading, so the cycle does not explode.

```
def validate_ingredients(ingredients: str) -> str:
    """Validate light-magic ingredients without a circular import."""
    from .light_spellbook import light_spell_allowed_ingredients

    lowered = ingredients.lower()
    allowed = light_spell_allowed_ingredients()
    is_valid = any(item in lowered for item in allowed)
    status = "VALID" if is_valid else "INVALID"
    return f"{ingredients} - {status}"
```

alchemy/grimoire/__init__.py

What it does: Exposes only the safe light_spell_record function through the grimoire package.

New thing to learn: A package interface can avoid importing dangerous modules automatically.

```
from .light_spellbook import light_spell_record

__all__ = ["light_spell_record"]
```

alchemy/grimoire/dark_spellbook.py

What it does: Defines dark ingredients and dark_spell_record, but imports dark_validator at file-load time.

New thing to learn: The first half of an intentional circular import.

```
from .dark_validator import validate_ingredients


def dark_spell_allowed_ingredients() -> list[str]:
    """Return ingredients accepted by dark magic."""
    return ["bats", "frogs", "arsenic", "eyeball"]


def dark_spell_record(spell_name: str, ingredients: str) -> str:
    """Record a dark spell when at least one ingredient is allowed."""
    validation = validate_ingredients(ingredients)
    if validation.endswith(" - VALID"):
        return f"Spell recorded: {spell_name} ({validation})"
    return f"Spell rejected: {spell_name} ({validation})"
```

alchemy/grimoire/dark_validator.py

What it does: Immediately imports dark_spell_allowed_ingredients back from dark_spellbook.

New thing to learn: The second half of an intentional circular import.

Beginner note: dark_spellbook -> dark_validator -> dark_spellbook. The second import asks for dark_spell_allowed_ingredients before dark_spellbook has finished defining it, so Python raises ImportError from a partially initialized module.

```
from .dark_spellbook import dark_spell_allowed_ingredients


def validate_ingredients(ingredients: str) -> str:
    """Validate dark ingredients with the intentional circular import."""
    lowered = ingredients.lower()
    allowed = dark_spell_allowed_ingredients()
    is_valid = any(item in lowered for item in allowed)
```

```
    status = "VALID" if is_valid else "INVALID"
    return f"{ingredients} - {status}"
```

ft_kaboom_0.py

What it does: Imports the safe grimoire package and records the light spell "Fantasy".

New thing to learn: Testing a package that avoids a circular import.

```
import alchemy.grimoire as grimoire


def main() -> None:
    """Record a light spell without a circular-import failure."""
    print("=== Kaboom 0 ===")
    print("Using grimoire module directly")
    result = grimoire.light_spell_record(
        "Fantasy",
        "Earth, wind and fire",
    )
    print(f"Testing record light spell: {result}")


if __name__ == "__main__":
    main()
```

ft_kaboom_1.py

What it does: Prints the warning, then imports dark_spellbook directly and intentionally crashes.

New thing to learn: Recognizing the runtime signature of a circular import.

Beginner note: The import is placed inside main() so the warning is printed before Python tries to load the intentionally broken dark modules.

```
def main() -> None:
    """Trigger the required circular-import failure for dark magic."""
    print("=== Kaboom 1 ===")
    print("Access to alchemy/grimoire/dark_spellbook.py directly")
    print("Test import now - THIS WILL RAISE AN UNCAUGHT EXCEPTION")

    from alchemy.grimoire.dark_spellbook import dark_spell_record

    print(dark_spell_record("Forbidden", "Bats and frogs"))


if __name__ == "__main__":
    main()
```

8. How to run and test the project

Run every command from the root of the project tree.

```
python3 ft_alembic_0.py
python3 ft_alembic_1.py
python3 ft_alembic_2.py
python3 ft_alembic_3.py
python3 ft_alembic_4.py      # expected AttributeError
python3 ft_alembic_5.py
python3 ft_distillation_0.py
python3 ft_distillation_1.py
python3 ft_transmutation_0.py
python3 ft_transmutation_1.py
python3 ft_transmutation_2.py
```

Study the imports, not only the final output.

```
python3 ft_kaboom_0.py
python3 ft_kaboom_1.py       # expected circular ImportError
```

Expected failures are part of the lesson: Do not "fix" ft_alembic_4 or the dark circular import if your goal is to match the supplied subject. Those failures demonstrate hidden package names and circular imports.

Run the required quality checks

```
python3 -m compileall .
flake8 .
mypy .
```

Note: the subject explicitly expects a mypy error for the hidden alchemy.create_earth access in ft_alembic_4.py. Explain why it is intentional.

9. Questions you should be able to answer

Q: What is a module?

A: A single Python file that can contain functions, classes, and variables.

Q: What is a package?

A: A folder used to organize related Python modules. In this project, alchemy is a package.

Q: Why does alchemy.create_air work?

A: alchemy/__init__.py imports create_air, so that name becomes available on the alchemy package.

Q: Why does alchemy.create_earth fail?

A: alchemy/__init__.py never exposes create_earth.

Q: What does as heal do?

A: It imports healing_potion but gives it the shorter package name heal.

Q: What does one dot mean in from .elements import ...?

A: Import from the current package.

Q: What do two dots mean in from ..potions import ...?

A: Go to the parent package, then import potions.

Q: Why does the light grimoire work?

A: The validator delays its import until validate_ingredients() is called, after the spellbook has loaded.

Q: Why does the dark grimoire fail?

A: Both files import each other at module-load time before the first module has finished defining its functions.

Q: How else can circular imports be fixed?

A: Move shared code into a third module, redesign dependencies so they point one way, or delay one import locally.

10. Beginner submission checklist

- Recreate the exact file tree at the repository root.
- Run every non-failing script and compare the important output with the subject examples.
- Confirm ft_alembic_4 fails for the hidden create_earth name.
- Confirm ft_kaboom_1 fails with a circular-import message.
- Run flake8 and fix normal style problems.
- Run mypy and understand the intentional ft_alembic_4 error.
- Do not add sys.path hacks.
- Do not use eval() or exec().
- Practice explaining import, from ... import ..., aliases, __init__.py, absolute/relative imports, and circular imports without looking at the code.
- Commit the expected tree to your Git repository before evaluation.

Fast study order: First master Alembic 0-5. Then understand how potions imports both element modules. Next trace recipes.py import paths. Finish by drawing the light and dark dependency arrows on paper.

Source basis: the user-supplied "The Codex - Mastering Python's Import Mysteries", Module 06, Version 2.0. Requirements and outputs are kept aligned with that subject.